UNIVERSIDAD SERGIO ARBOLEDA — ESCUELA DE CIENCIAS EXACTAS E INGENIERÍA
Ingeniería de Sistemas y Computación · Asignatura: Arquitectura de Software · Semestre 2026-2

Diseño de Gestión de Usuarios, Catálogo Botánico y Vinculación de Matera

Proyecto Matera Inteligente · SmartPlantBase | Versión Simplificada, Defendible y Libre de Sobreingeniería
Autor: Esteban (Ingeniería de Sistemas) · Apoyo: College Agent · Fecha: Septiembre 2026

---

1. Enfoque y Propósito del Documento

El objetivo fundamental de este documento es definir un modelo arquitectónico claro, directo y libre de sobreingeniería para satisfacer tres necesidades operativas esenciales del ecosistema SmartPlantBase:
1. Gestión Segura de Usuarios y Catálogo: Definir roles claros que garanticen la curaduría agronómica y protejan la base de conocimientos botánicos de datos caóticos o ficticios.
2. Precisión Conceptual de la Tríada: Modelar con rigurosidad la diferencia entre el hardware reutilizable (matera), el organismo biológico perecedero (planta/mata) y la plantilla taxonómica de referencia (especie).
3. Vinculación Física Transparente: Establecer un flujo simple mediante código QR impreso en el hardware, desacoplado de detalles de firmware o microcontrolador.

> 🛡️ **CRITERIO DE PROPORCIONALIDAD Y DEFENSA ORAL (ANTI-OVERENGINEERING GATE)**
> Este diseño elimina de raíz abstracciones prematuras que sobrecargaban versiones anteriores: criptografía asimétrica por hardware en microcontroladores, tokens de aprovisionamiento de fábrica a 128 bits, sincronización obligatoria RTC/NTP y particiones NVS en memoria flash.
> Criterio Rector: Un estudiante de ingeniería de sistemas debe ser capaz de dibujar este modelo en un tablero y defenderlo con total soltura en 5 minutos ante el docente examinador, demostrando maestría en Clean Architecture y proporcionalidad tecnológica.

2. Roles del Sistema y Gestión de Usuarios

Para mantener el sistema seguro, conciso y fácil de gobernar, se definen estrictamente dos roles sin jerarquías intermedias ni roles superfluos:

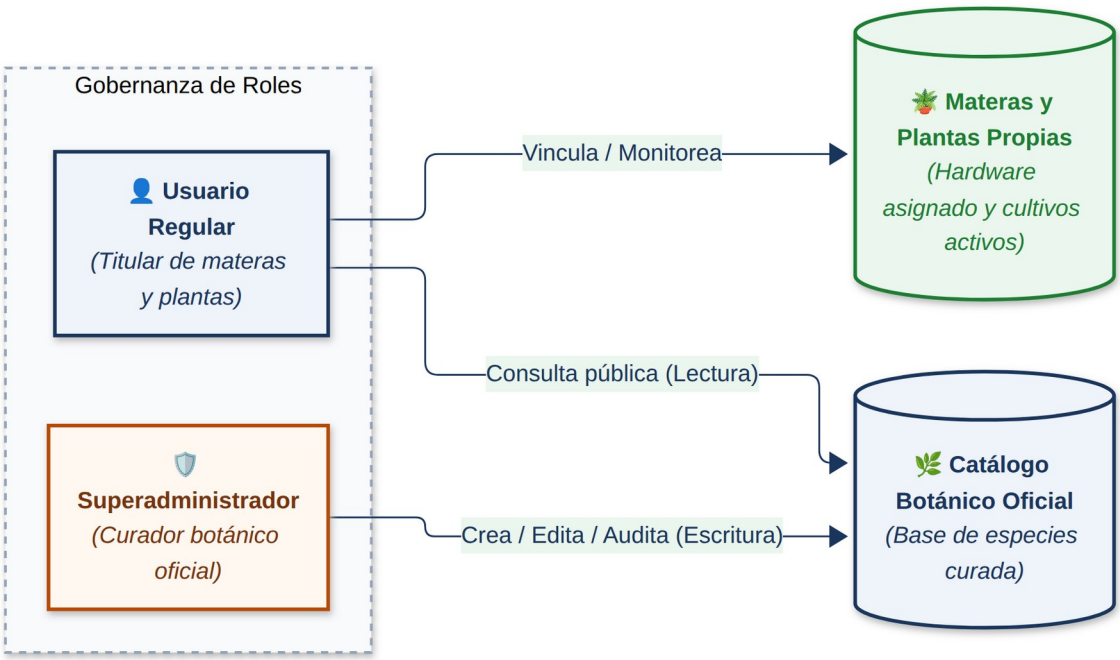


Figura 1: Arquitectura de Roles del Sistema — Curaduría Agronómica vs. Propiedad de Hardware.

Rol	Identificador	Alcance y Responsabilidad	Capacidades de Especies
Usuario Regular	**USER**	Dueño de sus materas físicas y plantas sembradas. Visualiza diagnósticos y alertas en su dashboard.	Lectura pública del catálogo. NO puede crear ni editar especies globales.
Superadministrador	**SUPERADMIN**	Curador agronómico y guardián de la base de datos oficial del sistema.	ÚNICO facultado para dar de alta nuevas especies botánicas y definir rangos.

2.1 El Usuario Regular (USER)

• Se registra mediante correo electrónico y contraseña (hasheada en almacenamiento seguro con salt).
• Es el titular exclusivo de sus materas físicas vinculadas y de las plantas vivas sembradas en ellas.
• Puede consultar en cualquier momento la lista de especies botánicas soportadas en la plataforma.
• Restricción fundamental de seguridad y dominio: El usuario regular NO puede registrar especies nuevas en el catálogo oficial de la plataforma.

2.2 El Superadministrador (SUPERADMIN)

• Usuario con privilegios elevados de curaduría agronómica y administración del sistema.
• Es el único actor facultado para invocar el endpoint protegido POST /api/v1/especies, creando fichas botánicas oficiales con sus rangos agronómicos de referencia (humedad %, iluminancia lux, temperatura °C).
• Puede corregir o normalizar parámetros ante nuevas publicaciones científicas o calibraciones de campo.

2.3 ¿Por qué es INDISPENSABLE un Superadmin para las Especies?

Esta separación no responde a un capricho administrativo, sino a una decisión crítica de arquitectura de software que previene dos fallos graves que quebrarían el funcionamiento de la aplicación:

> ⚙️ **JUSTIFICACIÓN DE INGENIERÍA: LOS 2 MOTIVOS CENTRALES DEL SUPERADMIN**
>
> 1. Problema del Catálogo Caótico (Fragmentación y Basura): Si los usuarios crearan libremente especies, en cuestión de días la base de datos se llenaría de duplicados desordenados: 'Tomate', 'tomate', 'Tomate cherry', 'tomatico' y 'Solanum lycopersicum'. Cada uno asociado a una matera distinta, haciendo inviable la analítica, las búsquedas cruzadas y la comparativa agronómica entre cultivos.
>
> 2. Problema de la Integridad Agronómica y el Motor de Reglas: El valor central de SmartPlantBase reside en su EvaluadorDiagnostico, el cual compara lecturas físicas contra rangos biológicos óptimos. Si un usuario ingresa una especie con límites absurdos (ej. temperatura óptima de 95°C o humedad del 2%), el motor generará diagnósticos fallidos o recomendaciones que ahoguen o sequen la planta. El Superadmin actúa como curador botánico oficial, garantizando la validez agronómica de la plataforma.

3. Modelo Conceptual: La Tríada Clave

Un error arquitectónico habitual en proyectos IoT es fusionar la matera física con la planta viva en una sola tabla o entidad. En el dominio real existen tres abstracciones con ciclos de vida totalmente disjuntos:

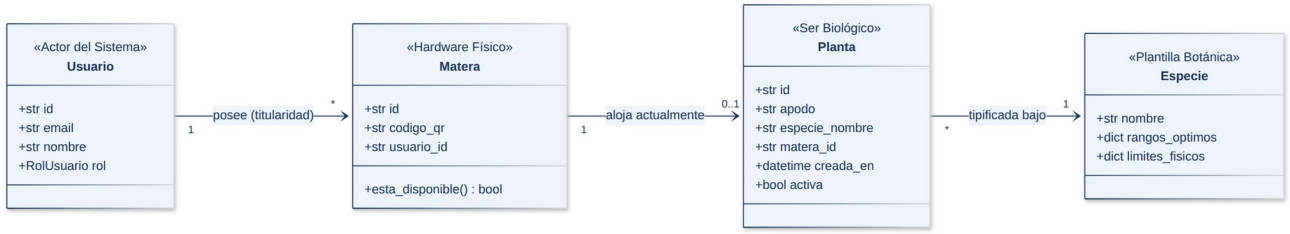


Figura 2: Modelo Conceptual de la Tríada Clave — Desacoplamiento de Hardware, Ser Vivo y Plantilla Botánica.

Concepto	¿Qué es en la realidad?	¿Quién lo administra?	Identificador	Ejemplo Práctico
Especie	Ficha técnica botánica con rangos agronómicos de supervivencia y bienestar.	Superadmin	Nombre botánico único	Albahaca (Ocimum basilicum), Sansevieria
Matera	Recipiente físico contenedor con circuito electrónico ESP32 empotrado.	Fabricante / Sistema	Código QR físico único	MAT-001, MAT-002, MAT-045
Planta / Mata	Organismo vivo sembrado en la matera con ciclo de vida dinámico.	Usuario Regular	ID autogenerado (UUID)	'Mi Albahaca de la cocina', 'Cactus de sala'

Estructura de Relaciones del Modelo:
• Usuario (1) posee muchas (*) Materas físicas vinculadas a su cuenta.
• Matera (1) contiene (0..1) Planta viva en un momento dado.
• Planta (*) es de tipo (1) Especie botánica del catálogo oficial curado.

> 🎓 **PREGUNTA CLAVE DE SUSTENTACIÓN ORAL ANTE EL DOCENTE**
>
> Profesor: '¿Por qué separaron Matera de Planta en entidades y tablas distintas en lugar de tener una sola tabla?'
>
> Respuesta Defensiva del Estudiante: 'Porque la matera es un activo físico de hardware reutilizable, mientras que la planta es un ser biológico perecedero. Si una planta se marchita o el usuario decide cosecharla, retira la tierra y siembra un cactus en la MISMA matera física. Al desacoplarlas, reutilizamos el hardware MAT-001 infinitas veces sin perder el histórico de diagnósticos de la planta anterior y sin tener que dar de baja el hardware del sistema.'

4. El ESP32: Principio de la Caja Negra

Para evitar reincidir en la sobreingeniería de arquitecturas descartadas, se define de manera rigurosa la frontera del microcontrolador:

> 📦 **REGLA DE ALCANCE: EL MICROCONTROLADOR COMO CAJA NEGRA**
>
> En esta etapa de gestión de usuarios, catálogos y vinculación, el ESP32 se trata estrictamente como una caja negra:
> • Sabemos que es el circuito físico empotrado dentro del plástico de la matera que lee los sensores analógicos y digitales.
> • NO modelamos en este nivel pines GPIO, rutinas de firmware C++, certificados TLS de silicio, timers FreeRTOS ni calibración de voltajes.
> • Para el backend de gestión, la matera física se identifica únicamente por su Código QR ('MAT-001'). La telemetría de lecturas se conectará en su respectivo hito como un adaptador de entrada más, manteniendo la pureza de la arquitectura.

5. Flujo de Vinculación con Código QR

5.1 Contenido del Código QR Físico

El código QR adhesivo pegado en la carcasa plástica de la matera contiene un identificador simple y legible:
• Formato directo texto plano: 'MAT-001'
• Formato JSON ligero: {"matera_id": "MAT-001"}
No se requieren esquemas criptográficos asimétricos ni claves secretas de 128 bits para el alcance universitario del proyecto.

5.2 Secuencia de Vinculación Paso a Paso

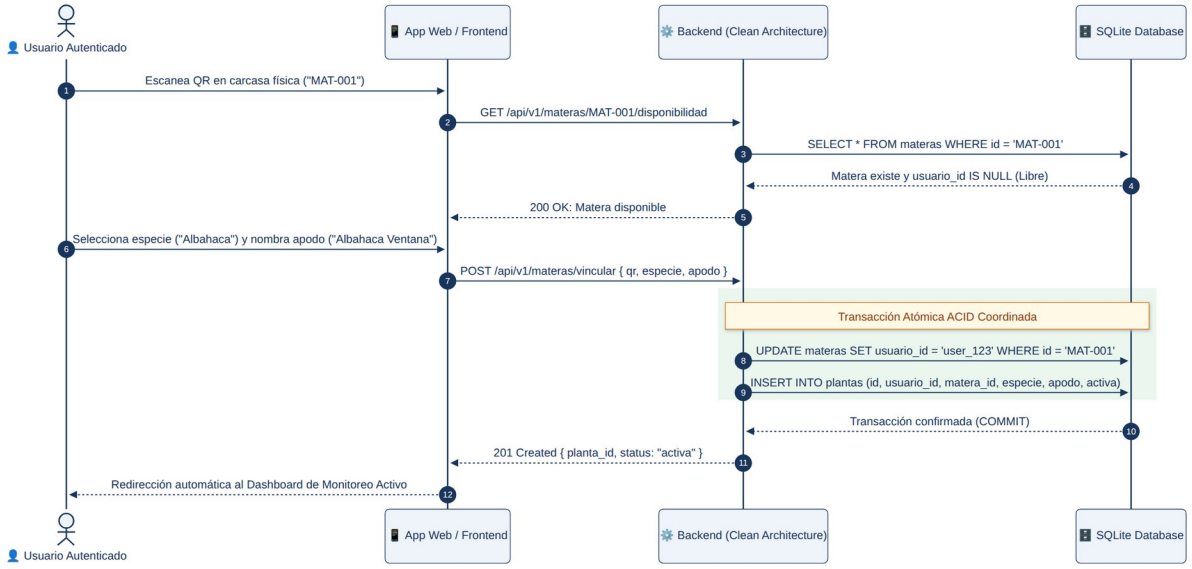


Figura 3: Diagrama de Secuencia — Flujo Atómico de Vinculación de Matera mediante Código QR.

Paso	Interacción / Endpoint	Lógica de Negocio y Persistencia
1	**Escaneo QR en App Web/Móvil**	La cámara decodifica el string 'MAT-001' adherido a la carcasa de la matera física.
2	**GET /api/v1/materas/MAT-001/disponibilidad**	El backend verifica en SQLite si MAT-001 existe y si su usuario_id es NULL (está libre). Retorna 200 OK disponible.
3	**Selección en Interfaz**	El usuario elige la especie del catálogo oficial (ej. 'Albahaca') y le asigna un apodo (ej. 'Albahaca Ventana').
4	**POST /api/v1/materas/vincular**	Transacción atómica: 1) Asigna matera.usuario_id = usuario_id; 2) Inserta nuevo registro de Planta activa vinculada a la matera y a la especie.
5	**201 Created & Renderizado**	El backend confirma la vinculación y la app redirige al dashboard con la planta activa en monitoreo.

5.3 Reglas de Negocio en la Vinculación

1. Matera Inexistente: Si el código QR leído no existe en la base de datos, el backend responde 404 Not Found ('Matera no registrada en el sistema').
2. Matera Ocupada: Si la matera ya tiene un usuario_id asignado distinto al usuario autenticado, el sistema rechaza la vinculación con 409 Conflict ('Matera ya vinculada a otro usuario').
3. Especie Válida: La especie seleccionada debe existir previamente en el catálogo botánico oficial (EspecieRepository).

4. Desvinculación Limpia: Si el usuario desvincula la matera, la planta pasa a estado inactiva (preservando histórico) y el campo matera.usuario_id vuelve a NULL, disponible para otro usuario.

6. Alineación con Clean Architecture (SmartPlantBase)

El diseño respeta plenamente la arquitectura limpia ya implementada en el repositorio SmartPlantBase, sin alterar el núcleo de reglas de negocio previamente validado:

📄 Estructura del Proyecto SmartPlantBase (Clean Architecture)

```
src/
├── domain/                 # Reglas de negocio puras (Python puro, 0 dependencias externas)
│   ├── entities.py          # Invarianza: Se preservan Medicion, Especie, Diagnostico.
│   │                        # Nuevas entidades: Usuario, RolUsuario, Matera, Planta.
│   ├── rules.py             # EvaluadorDiagnostico (INTACTO, 0 modificaciones).
│   └── repositories.py       # Interfaces: EspecieRepo, UsuarioRepo, MateraRepo, PlantaRepo.
├── application/             # Casos de uso de orquestación
│   ├── dtos.py              # DTOs tipados de entrada y salida.
│   └── use_cases.py          # RegistrarUsuario, Autenticar, CrearEspecie, VincularMatera.
├── infrastructure/          # Adaptadores de persistencia y drivers
│   └── repositories/        # SQLiteRepositories (tablas relacionales livianas).
└── presentation/            # Controladores HTTP (Flask)
    └── controllers.py       # Rutas públicas y protegidas con sesión de usuario.
```

6.1 Nuevas Entidades de Dominio (src/domain/entities.py)

Implementadas con dataclasses estándar de Python sin acoplamiento a librerías externas:

📄 src/domain/entities.py — Extensiones de Dominio

```
from dataclasses import dataclass
from enum import Enum
from typing import Optional
from datetime import datetime

class RolUsuario(str, Enum):
    USER = "USER"
    SUPERADMIN = "SUPERADMIN"

@dataclass
class Usuario:
    id: str
    email: str
    password_hash: str
    nombre: str
    rol: RolUsuario = RolUsuario.USER
    activo: bool = True

    def es_superadmin(self) -> bool:
        return self.rol == RolUsuario.SUPERADMIN

@dataclass
class Matera:
    id: str                # Código legible: "MAT-001"
    codigo_qr: str         # String decodificado del QR
    usuario_id: Optional[str] = None  # None = Disponible

    @property
    def esta_disponible(self) -> bool:
        return self.usuario_id is None

@dataclass
class Planta:
    id: str
    usuario_id: str
    matera_id: str
    especie_nombre: str
    apodo: str
    creada_en: datetime
    activa: bool = True
```

> **🌱 INVARIANZA DEL NÚCLEO BOTÁNICO (PRINCIPIO ABIERTO/CERRADO)**
> Las clases Medicion, Diagnostico, LimiteFisico y el motor EvaluadorDiagnostico no sufren una sola modificación. Gracias a Clean Architecture, la incorporación de usuarios, roles y hardware se realiza de manera perimetral sin alterar la lógica de negocio probada con pruebas unitarias.

6.2 Casos de Uso Clave (src/application/use_cases.py)

1. RegistrarUsuarioUseCase & AutenticarUsuarioUseCase: Registro y validación simple de credenciales con hash seguro.
2. CrearEspecieUseCase: Recibe el ID del solicitante; si su rol no es SUPERADMIN, rechaza la operación con PermisoDenegadoError. Si es superadmin, valida rangos biológicos mediante LIMITES_FISICOS y almacena la nueva ficha en EspecieRepository.
3. VincularMateraConQRUseCase: Valida existencia y disponibilidad de la matera, existencia de la especie, asigna matera.usuario_id y persiste la nueva Planta viva en una sola transacción atómica coordinada.

4. ListarPlantasDeUsuarioUseCase: Recupera las plantas activas del usuario autenticado con su estado de telemetría y matera asociada.

6.3 Modelo Relacional Simple en SQLite

Esquema DDL relacional claro, sin complejidades de servidores de bases de datos externos:

📄 database/schema.sql — Esquema Relacional Autocontenido

```
-- Usuarios del sistema
CREATE TABLE usuarios (
    id TEXT PRIMARY KEY,
    email TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    nombre TEXT NOT NULL,
    rol TEXT NOT NULL DEFAULT 'USER',
    activo INTEGER NOT NULL DEFAULT 1
);

-- Materas físicas (con hardware ESP32 empotrado)
CREATE TABLE materas (
    id TEXT PRIMARY KEY,
    codigo_qr TEXT UNIQUE NOT NULL,
    usuario_id TEXT NULL,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)
);

-- Plantas vivas de los usuarios
CREATE TABLE plantas (
    id TEXT PRIMARY KEY,
    usuario_id TEXT NOT NULL,
    matera_id TEXT NOT NULL,
    especie_nombre TEXT NOT NULL,
    apodo TEXT NOT NULL,
    creada_en TEXT NOT NULL,
    activa INTEGER NOT NULL DEFAULT 1,
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id),
    FOREIGN KEY (matera_id) REFERENCES materas(id)
);
```

Ventajas de Persistencia: SQLite almacena todo el estado en un archivo local autocontenido, facilitando la reproducibilidad, las pruebas automatizadas y la defensa en vivo del proyecto sin dependencias de infraestructura.

7. Guía Ejecutiva de Sustentación Oral (5 Minutos en Tablero)

Al sustentar este diseño ante el docente, el estudiante debe dibujar el siguiente esquema en el tablero y responder con convicción a las cuatro preguntas críticas de arquitectura:

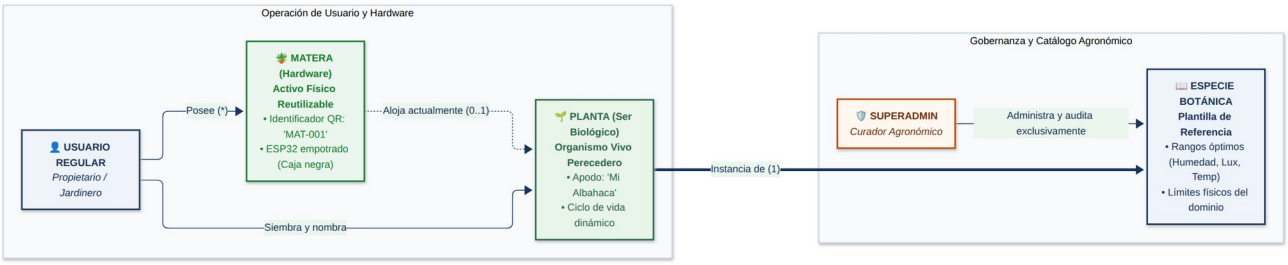


Figura 4: Mapa Conceptual para Sustentación Oral en Tablero — Gobernanza de Catálogo vs. Hardware de Usuario.

🎓 LAS 4 RESPUESTAS CLAVE PARA LA DEFENSA ORAL ANTE EL DOCENTE

1. '¿Por qué un Superadmin para las especies?': Para blindar la integridad agronómica del catálogo. Si cualquier usuario crea especies, se colapsa la base de datos con duplicados inconsistentes ('tomate', 'tomatico', 'Tomate cherry') y valores fuera de la realidad biológica que provocarían fallos graves en el motor de diagnóstico.
2. '¿Por qué Matera no es lo mismo que Planta?': Porque la matera es el hardware físico reutilizable (recipiente y circuito) y la planta es el ser vivo perecedero. Si la planta muere o se retira, la matera física se reutiliza para sembrar otra especie sin perder el histórico de diagnósticos ni tener que dar de baja el hardware.
3. '¿Cómo funciona la vinculación por QR?': El QR contiene únicamente el identificador legible ('MAT-001'). La aplicación valida que esté disponible (usuario_id es NULL), y en una sola transacción asocia la matera al usuario y crea la planta con su especie oficial y apodo.
4. '¿Y dónde queda el ESP32 en este diseño?': El ESP32 es el componente de sensado dentro de la matera. En este nivel de software se trata como una caja negra que se integrará limpiamente como un adaptador de telemetría sin acoplar la gestión de usuarios a complejidades de silicio.